

Fiori Elements 開發課程 9：純 Report (無 BDEF)

環境：BTP ABAP Environment Trial (套件 ZRAPCLOUD)

Lecture

這一課要解決的問題

fe01 ~ fe08 用過的所有 CDS View，全部都有 BDEF——完整的 Create / Update / Delete / Action。但很多真實需求只是「給我看一份彙總報表」，完全不需要交易能力。這一課要示範**完全不需要 BDEF 的純 Report CDS View**：ZI_RC01_TASK_SUMMARY，依狀態彙總 ZI_RC01_TASK 的資料，是一個純唯讀的分析報表。

RAP Cloud 課程 rc08 的講義曾經提過「純 Report CDS View 不需要 root/BDEF 的完整步驟」，但當時只有文字性方法論說明，用的是佔位符 ZI_RCxx_REPORT，**沒有真的建立過對應物件**——這一課把它真正做出來。

CDS View 設計

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@Metadata.allowExtensions: true
@EndUserText.label: 'RC01 Task Status Summary (Report)'
define view entity ZI_RC01_TASK_SUMMARY
  as select from zrc01_task
  {
    key status,

    count(*) as TaskCount,
    sum( case when due_date < $session.system_date
            then 1
            else 0
          end ) as OverdueCount
  }
  group by status
```

\$session.system_date 是 CDS 內建 Session 變數 (取得當下日期)，count(*) / sum(case when...) 是標準 CDS 聚合語法，依 status 分組統計每個狀態有幾筆 Task、幾筆逾期。

⚠️⚠️ 為什麼不宣告 root

查證官方 Glossary 確認：「**Root Entity** — 是一個 **Business Object** 結構裡最上層的實體，用 **ROOT** 關鍵字宣告」。root 這個關鍵字語意上是「這是某個 Business Object 結構的最上層」——而「Business Object 結構」這個概念，只有掛了 BDEF 才存在 (BDEF 定義了交易行為：Create/Update/Delete、Lock、Draft、Composition 底下有哪些 Child Entity)。純 Report View **完全沒有 BDEF**，沒有交易行為、沒有 Composition 階層，沒有「結構」可言，自然也沒有「最上層」這回事——宣告 root 等於在暗示這裡有一個 Business Object 結構，但實際上根本不存在。

對照 fe07 讀過的 `C_SlsDocFlflmntAnalyzer` (Track Sales Orders 背後的純 Report View) ——它同樣沒有 `@ObjectModel.compositionRoot: true` (V1 語法對應 `root` 概念的寫法) ，印證了這個判斷。

結論：純 Report 用單層 `define view entity` (不是 `define root view entity`) ，也不需要 fe08 那套 Interface/Projection 兩層分離——因為兩層架構存在的意義是「把 Behavior 跟 UI 曝露分開」，沒有 Behavior 就沒有分開的必要。

Metadata Extension 與 Service

```
@Metadata.layer: #CUSTOMER
@UI.headerInfo: { typeName: 'Task Status Summary', typeNamePlural: 'Task Status Summary' }
annotate entity ZI_RC01_TASK_SUMMARY with
{
  @UI.lineItem: [ { position: 10 } ]
  @UI.selectionField: [ { position: 10 } ]
  status;

  @UI.lineItem: [ { position: 20 } ]
  TaskCount;

  @UI.lineItem: [ { position: 30 } ]
  OverdueCount;
}
```

```
define service ZRC01_TASKSUM_SD {
  expose ZI_RC01_TASK_SUMMARY as TaskSummary;
}
```

Service Definition 命名沒有沿用 `ZRCnn_SD` 慣例——`ZRC01_TASKSUM_SD` 而不是類似 `ZRC09_SD` 的寫法，因為這不是「rc01 本體的 Service」，是衍生的分析報表，用 `ZRCnn` 命名容易誤會成暗示不存在的「rc09」課程。

Eclipse ADT 建立步驟 (套件 ZRAPCLOUD)

1. **CDS View**：對套件右鍵 → `New Data Definition`，Name 填 `ZI_RC01_TASK_SUMMARY`，Template 選 `Define View`，貼入程式碼——⚠ 樣板如果預設帶 `root` 關鍵字要手動刪掉，存檔→啟用。這一步不需要 (也不能) 建立 **Behavior Definition**——右鍵選單如果出現這個選項，忽略它，這正是這一課要示範的重點。
2. **Metadata Extension**：對 `ZI_RC01_TASK_SUMMARY` 右鍵 → `New Metadata Extension`，Name 同名，貼入程式碼，存檔→啟用
3. **Service Definition**：對套件右鍵 → `New Service Definition`，Name 填 `ZRC01_TASKSUM_SD`，貼入程式碼，存檔→啟用
4. **Service Binding**：對 `ZRC01_TASKSUM_SD` 右鍵 → `New Service Binding`，Name 填 `ZRC01_TASKSUM_SB`，Binding Type 選 `OData V4 - UI`，選 `Transport Request` → 先啟用 (`Ctrl+F3`) → 再 **Publish**

⚠ 踩坑記錄

- 一開始漏加 `@Metadata.allowExtensions: true`，建 **Metadata Extension** 時報錯「**Annotation 'Metadata.allowExtensions' missing**」——這個旗標不限於「擴充別人的標準物件」才需要（fe07 的情境），任何 **CDS View** 只要想掛自己的 **Metadata Extension** 都要有它，這一課才發現這個更廣泛的規則。
- Service Binding 一樣要先啟用才能正常 Publish（跟 fe08 同一個坑）。

前端：VS Code Fiori Generator Step by Step

1. `Ctrl+Shift+P` → **Fiori: Open Application Generator**
2. Template 選 **List Report Page**
3. Data Source 選 **Connect to a System** → System 選既有的 **TRL**
4. **Service** 搜尋 `ZRC01_TASKSUM_SB`，選 `ZRC01_TASKSUM_SB > ZRC01_TASKSUM_SD (0001)`
5. 「Download value help metadata」選 **Yes**
6. Main Entity 選 **TaskSummary**，Table Type 選 **Responsive**
7. Project Attributes：Module Name 填 **fe09tasksummary**（⚠ 全小寫，見 fe08 記錄的「Module cannot contain capital letters」規則），Enable TypeScript：**No**，Project Folder Path 指到 `src/ABAP_Training_Fiori_Elements/`，**Finish**
8. 開終端機，先 `cd` 進 `src/ABAP_Training_Fiori_Elements/fe09tasksummary/` 這個專案資料夾（`npm start` 讀當下目錄的 `package.json`，必須先切換過去），再執行 `npm start`，等終端機印出 **Server started / URL: http://localhost:XXXX** 才是就緒訊號——**Port** 不保證是 **8080**，要看實際印出的號碼（fe01 ~ fe08 留下的專案如果還在跑，會自動往下一個空號遞增）
9. 瀏覽器自動開啟該 Port 的 `/test/flp.html#app-preview`；如果卡在瀏覽器原生 Basic Auth 帳密框，把該 Port 的 process 砍掉重開（fe04 講義記錄過的已知認證異常）

可以跟 **fe08** 的開發伺服器同時開著，各自 Port 不同，方便直接切分頁對照兩邊畫面差異（有 BDEF vs 沒有 BDEF）

驗證重點：List Report 表格上**不應該**看到 Create / Delete 按鈕，也不會有 Editing Status / Draft 相關的篩選欄位——這是「完全沒有 BDEF」最直接的畫面證據，跟 fe08 的畫面（有 Mark Done / Create / Delete）形成鮮明對照。

學習目標

- 知道純 Report CDS View（無 BDEF）是官方認可的合法模式，不是「偷懶」或「沒做完整」
- 能講出「為什麼不宣告 `root`」的根本原因：`root` 語意上綁定 Business Object 結構，沒有 BDEF 就沒有結構
- 能寫出 GROUP BY 彙總的 CDS View 語法（`count(*)`、`sum(case when...)`、`$session.system_date`）
- 知道 `@Metadata.allowExtensions: true` 是任何要掛 Metadata Extension 的 CDS View 的通用前提，不限於擴充標準物件
- 能對照 fe08（完整兩層 + BDEF）與這一課（單層無 BDEF），講出兩種模式分別適合什麼情境

物件清單

套件 **ZRAPCLOUD**：

物件	型別	說明
ZI_RC01_TASK_SUMMARY	DDL	define view entity (無 root) , GROUP BY status 彙總
ZI_RC01_TASK_SUMMARY	DDLX (Metadata Extension)	UI Annotation
ZRC01_TASKSUM_SD	Service Definition	expose ZI_RC01_TASK_SUMMARY as TaskSummary;
ZRC01_TASKSUM_SB	Service Binding (OData V4-UI)	已 Publish

沒有 BDEF、沒有 Behavior Implementation Class——這正是這一課要證明的事。

動手練習

輪到你了：

1. 幫這個 Report 加一個依 `priority` 再細分的維度 (`GROUP BY status, priority`) , 想一想欄位清單跟 Metadata Extension 要怎麼跟著調整
2. 試著在 Fiori Launchpad 打開這個 App 的 List Report , 確認畫面上有沒有出現 Create / Delete 按鈕 (提示：因為沒有 BDEF , 答案很明確 , 但親自看一次會更有印象)
3. 想一想：如果之後這份報表想要能篩選日期區間 (例如只看某段期間建立的 Task) , 要在 CDS View 加什麼？ (提示：查一下 CDS Parameter , `with parameters` 語法)

驗證方式

後端物件已在 Eclipse 完整建立、啟用、Service Binding 已 Publish 成功 (截圖為證 , 過程中排查了 `@Metadata.allowExtensions` 遺漏的問題) 。前端 Fiori Elements App 的產生與畫面驗證見下一節動手操作 (沿用 fe01 的 VS Code Generator 流程 , Service 選 `ZRC01_TASKSUM_SB` > `ZRC01_TASKSUM_SD` → `TaskSummary`) , 驗證重點是**確認畫面上沒有 Create/Edit/Delete 按鈕、沒有 Draft 相關 UI** (Editing Status 篩選、Discard/Activate 之類) , 因為底層完全沒有 BDEF 提供這些能力。

思考題

1. 這一課的 `OverdueCount` 是用 `sum(case when...)` 算出來的「動態聚合」, 如果之後資料量很大, 這種即時聚合的效能會不會是問題? 有沒有更適合大量資料的做法? (提示：想一想 HANA 的 Analytical Query / Cube 相關概念, 這是進階題, 不要求做出來)
2. fe08 跟這一課分別代表「完整兩層 + BDEF」跟「單層無 BDEF」兩個極端。實務上你覺得多數的自訂開發會落在哪裡? 什麼情境會讓你選擇「單層 + 簡單 BDEF」(像 rc01 原本的 `ZI_RC01_TASK`) 這種介於中間的做法?

答案

見 `ZI_RC01_TASK_SUMMARY` (DDL / DDLX) 、 `ZRC01_TASKSUM_SD` 、 `ZRC01_TASKSUM_SB` , 皆建立於 BTP Trial 系統套件 `ZRAPCLOUD` 。